

RAG - Knowledge Management (Local Setup)

Naresh Chaurasia

Version 1.3, 2025-07-13: Fourth Edition - Added Week 5 final integrative capstone

Table of Contents

1. Overview	1
2. Prerequisites	2
3. Architecture Overview	3
4. Step-by-Step Setup	4
4.1. Python Virtual Environment	4
4.2. Install PostgreSQL 18 with pgvector	4
4.3. Create the Database and Enable pgvector	5
4.4. Initialize the Database Table	5
4.5. Set a Password	6
4.6. View Logs	6
4.7. Disable Automatic Startup	6
4.8. Uninstall PostgreSQL	6
4.9. Project-Specific: Quick Verification Checklist	7
4.10. Useful Commands	7
4.11. Install Redis (for Semantic Caching)	7
4.12. Update the .env File	8
4.13. Install Python Dependencies	8
4.14. How the Code Calls Dependencies (Important)	9
4.15. Run the Application	9
4.16. Open the Web UI	11
4.17. Ingest the Documents	11
4.18. □ Ask Questions	12
4.19. □ (Optional) Run RAGAS Evaluation	12
4.20. Clean Database and Re-run Ingestion	12
5. File-by-File Explanation	14
5.1. <code>docker-compose.yml</code>	14
5.2. <code>Dockerfile</code>	14
5.3. <code>.env</code>	15
5.3.1. □ Where LangSmith is Actually Used	15
5.4. <code>app/api.py</code> — FastAPI Application	16
5.4.1. <code>GET /</code>	16
5.4.2. <code>POST /ingest</code>	16
5.4.3. <code>GET /ingest/status</code>	16
5.4.4. <code>POST /ask</code>	16
5.5. <code>app/ingest.py</code> — Document Ingestion Pipeline	16
5.5.1. Step 1: Load Documents (<code>_load_docs()</code>)	16
5.5.2. Step 2: Chunk Documents (<code>_chunk()</code>)	17
5.5.3. Step 3: Store Embeddings	17

5.5.4. Step 4: Create Vector Index (<code>_create_index()</code>)	17
5.6. <code>app/rag.py</code> — RAG Retrieval + Generation Chain	17
5.6.1. System Prompt	17
5.6.2. Chain Construction (<code>_build_chain()</code>)	17
5.6.3. □ When Does Reranking Happen? Is it a Separate API Call?	18
5.6.4. Semantic Caching (DISABLED)	19
5.6.5. <code>answer_with_docs_async()</code>	20
5.7. <code>app/utils.py</code> — Database & Vector Store Setup	20
5.8. <code>app/eval_ragas.py</code> — RAGAS Evaluation	20
5.9. <code>app/static/index.html</code> — Frontend UI	20
5.10. <code>app/static/style.css</code>	21
5.11. <code>init-db/init.sql</code>	21
5.11.1. What is <code>langchain_metadata</code> Used For?	21
5.12. <code>seed/qna_test.json</code>	22
5.13. <code>data/</code> — Sample Documents	23
5.14. <code>pgvector</code>	23
5.15. Verifying Everything is Running	24
5.16. Stopping the Application	24
5.17. Restarting After Stopping	25
6. Troubleshooting	26
6.1. <code>ModuleNotFoundError: No module named 'langchain_classic'</code>	26
6.2. <code>ModuleNotFoundError: No module named 'langchain_postgres'</code>	26
6.3. Connection refused to PostgreSQL	26
6.4. Connection refused to Redis	26
6.5. Ingestion fails with authentication error	26
7. Summary of Services and Ports	28
8. Architecture Diagram	29
9. Dependencies	30
10. Key Design Decisions	31
11. Notes	32
12. Q&A	33

Chapter 1. Overview

This guide explains how to run the **Company Knowledge Assistant** RAG application **without Docker** — directly on your macOS machine with locally installed PostgreSQL, Redis, and Python.

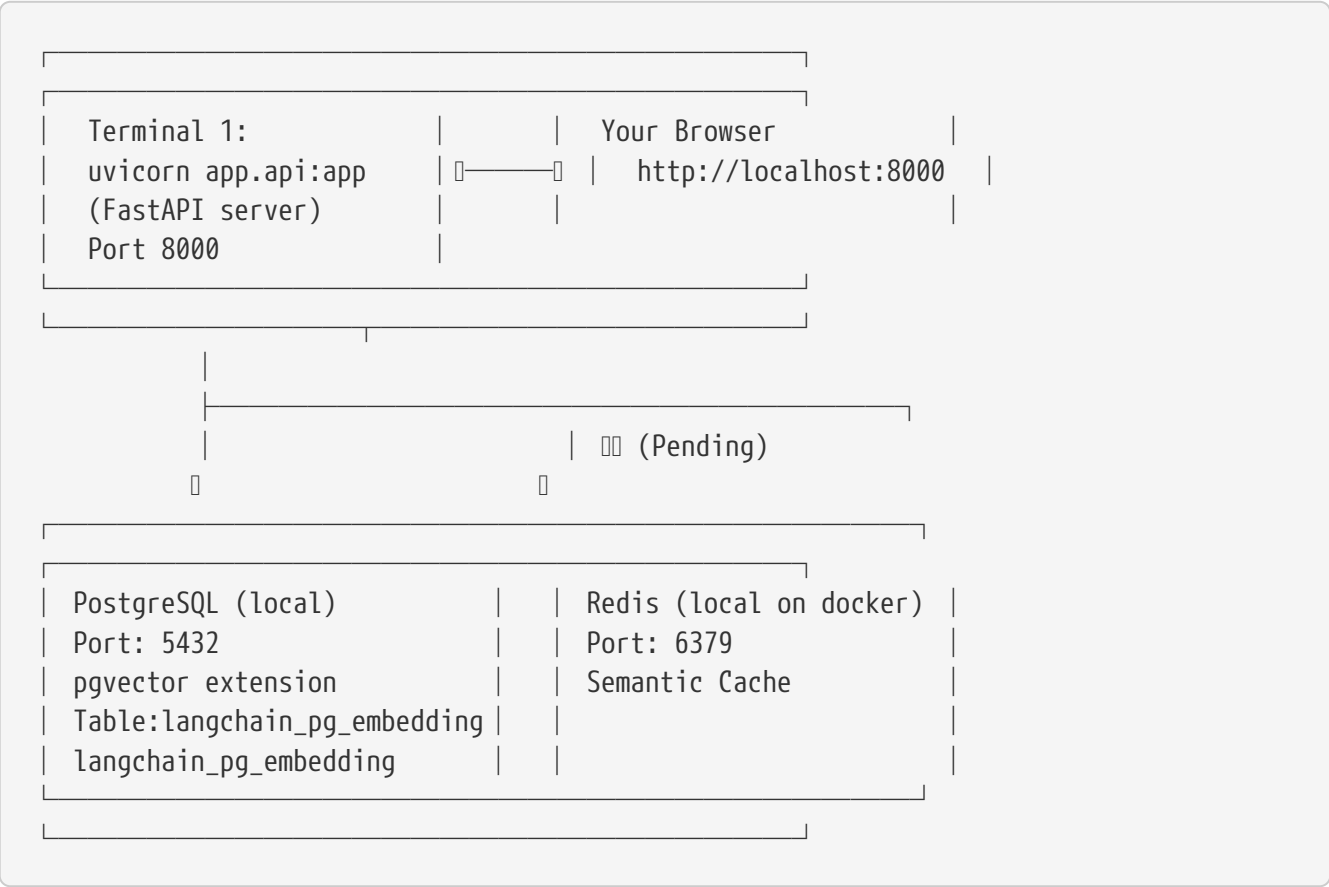
Chapter 2. Prerequisites

Before starting, ensure you have the following installed on your machine:

- **Homebrew** (macOS package manager) — <https://brew.sh>
- **Python 3.11+** — Check with `python3 --version`
- **PostgreSQL 17** (with pgvector extension)
- **Redis**
- An **OpenAI API key** (with access to `text-embedding-3-small` and `gpt-4o-mini`)
- (Optional) A **Cohere API key** for reranking

Chapter 3. Architecture Overview

When running locally without Docker, your system runs these processes:



Chapter 4. Step-by-Step Setup

4.1. Python Virtual Environment

Open a terminal and run:

```
# All environments will be created in this directory:
cd /Users/nareshchaurasia/nc/PYTHON-ARCHITECT/Python-Immersive-AI/ALL-VEENV

python3 -m venv .venv_knowledge_management

source .venv_knowledge_management/bin/activate

# Change to the project directory
cd /Users/nareshchaurasia/nc/PYTHON-ARCHITECT/Python-Immersive-AI/Frontier-Engineer/week2/e2e-projects/rag-knowledge-management
```

Your prompt should now show `(venv_knowledge_management)` indicating the virtual environment is active.

4.2. Install PostgreSQL 18 with pgvector

```
# Install PostgreSQL 18
brew install postgresql@18

# Start PostgreSQL as a background service
brew services start postgresql@18

# Start manually if you prefer not to use brew services
/opt/homebrew/opt/postgresql@18/bin/pg_ctl \
  -D /opt/homebrew/var/postgresql@18 \
  -l /opt/homebrew/var/postgresql@18/server.log start

# Install pgvector extension
brew install pgvector

# Verify PostgreSQL is running
pg_isready
Expected output: `/tmp/.s.PGSQL.5432 - accepting connections`
```

4.3. Create the Database and Enable pgvector

```
# Connect to PostgreSQL as the default user (your macOS username)
psql -d postgres

# Inside the psql shell, run these commands one by one:
CREATE EXTENSION IF NOT EXISTS vector;
\q
```

NOTE: If your macOS username is different from `postgres`, you may need to create the `postgres` user or update the connection string later. Run this if needed:

```
createuser -s postgres
psql -d postgres -c "\du"
```

4.4. Initialize the Database Table

```
psql -U postgres -d postgres -f init-db/init.sql
```

This creates the `langchain_pg_embedding` table with the vector column.

Verify the table was created:

```
psql -U postgres -d postgres -c "\d langchain_pg_embedding"
```

You should see columns: `langchain_id`, `content`, `embedding`, `langchain_metadata`, `category`.

```
langchain_metadata:
{
  "page": 5,
  "title": "Prompting Techniques for Large Language Models",
  "author": "Naresh Chaurasia",
  "format": "PDF 1.4",
  "source": "data/frontier-engineering/week1/010-Prompting-Techniques.pdf",
  "creator": "Naresh Chaurasia",
  "modDate": "D:20260618173445+05'00'",
  "moddate": "2026-06-18T17:34:45+05:00",
  "subject": "",
  "trapped": "",
  "keywords": "",
  "producer": "Asciidoctor PDF 2.3.24, based on Prawn 2.4.0",
  "file_path": "data/frontier-engineering/week1/010-Prompting-Techniques.pdf",
```



```
"total_pages": 22,  
"creationDate": "D:20260618173445+05'00'",  
"creationdate": "2026-06-18T17:34:45+05:00"  
}
```

4.5. Set a Password

```
ALTER USER nareshchaurasia  
WITH PASSWORD 'Hello12#$';
```

After setting a password, update the `.env DATABASE_URL` to include it:

```
DATABASE_URL=postgresql+asyncpg://nareshchaurasia:Hello12#$@localhost:5432/postgres
```

4.6. View Logs

```
tail -50 /opt/homebrew/var/postgresql@17/server.log
```

4.7. Disable Automatic Startup

If you don't want PostgreSQL to start automatically at login:

```
brew services stop postgresql@17
```

```
launchctl bootout gui/${id -u} \  
~/Library/LaunchAgents/homebrew.mxcl.postgresql@17.plist  
  
rm ~/Library/LaunchAgents/homebrew.mxcl.postgresql@17.plist
```

4.8. Uninstall PostgreSQL

```
brew services stop postgresql@17  
brew uninstall postgresql@17  
rm -rf /opt/homebrew/var/postgresql@17  
brew cleanup
```

4.9. Project-Specific: Quick Verification Checklist

Run these commands to verify PostgreSQL is properly set up for the project:

```
# 1. PostgreSQL is running
pg_isready

# 2. pgvector extension is enabled
psql -U postgres -d postgres -c "SELECT * FROM pg_extension WHERE extname='vector';"

# 3. The embeddings table exists with the correct schema
psql -U postgres -d postgres -c "\d langchain_pg_embedding"

# 4. The HNSW index exists (after ingestion)
psql -U postgres -d postgres -c "\di langchain_pg_embedding_hnsw_idx"
```

4.10. Useful Commands

```
psql --version
brew services list
pg_isready
psql -d postgres
pg_ctl -D /opt/homebrew/var/postgresql@17 start
pg_ctl -D /opt/homebrew/var/postgresql@17 stop
tail -50 /opt/homebrew/var/postgresql@17/server.log
psql -U postgres -d postgres -c "\d langchain_pg_embedding"
psql -U postgres -d postgres -c "\di"
```

4.11. Install Redis (for Semantic Caching)

```
# Install Redis
brew install redis

# Start Redis as a background service
brew services start redis

# Verify Redis is running
```

```
redis-cli ping
```

Expected output: **PONG**

4.12. Update the .env File

Open the **.env** file in the project directory and make these changes:

1. Replace the **DATABASE_URL** value with your local connection string:

```
DATABASE_URL=postgresql+asyncpg://postgres:postgres@localhost:5432/postgres
```

If you are using your macOS username instead of **postgres:**

```
DATABASE_URL=postgresql+asyncpg://YOUR_MACOS_USERNAME@localhost:5432/postgres
```

1. Replace the **REDIS_URL** value to point to localhost:

```
REDIS_URL=redis://localhost:6379/0
```

1. Update the **OPENAI_API_KEY** with your own key:

```
OPENAI_API_KEY=sk-your-actual-openai-api-key-here
```

1. (Optional) Update the **CO_API_KEY** with your Cohere key if you want reranking:

```
CO_API_KEY=your-cohere-api-key-here
```



The **.env** file currently contains live API keys. Replace them with your own keys to avoid hitting rate limits or unauthorized usage.

4.13. Install Python Dependencies

Create a **requirements.txt** file in the project root (**Frontier-Engineer/week2/e2e-projects/rag-knowledge-management/requirements.txt**) with the following content:

- **/Users/nareshchaurasia/nc/PYTHON-ARCHITECT/Python-Immersive-AI/ALL-VEENV/venv_knowledge_management_requirements.txt**

Then install all dependencies:

```
pip install --upgrade pip
pip install -r venv_knowledge_management_requirements.txt
```

Installation time: This may take 2–5 minutes depending on your internet connection.



If you encounter issues with `unstructured` or `pymupdf`, try installing missing system libraries:

```
brew install poppler tesseract libffi
```

Then retry: `pip install -r requirements.txt`

4.14. How the Code Calls Dependencies (Important)

The project uses these import paths — make sure they match what's installed:

- `langchain_classic` → provided by the `langchain-classic` package (used in `rag.py`)
- `langchain_postgres.v2` → provided by the `langchain-postgres` package (used in `utils.py` and `ingest.py`)

If you get `ModuleNotFoundError` for `langchain_classic`, run:

```
pip show langchain-classic
```

If not installed, install it: `pip install langchain-classic`

4.15. Run the Application

Make sure your virtual environment is still active (source `.venv_knowledge_management/bin/activate` if not).

```
cd Frontier-Engineer/week2/e2e-projects/rag-knowledge-management
uvicorn app.api:app --host 0.0.0.0 --port 8000 --reload
```

You should see output like:

```
INFO:      Uvicorn running on http://0.0.0.0:8000
INFO:      (Press CTRL+C to quit)
INFO:      Started reloader process [xxxxx] using WatchFiles
INFO:      Started server process [xxxxx]
INFO:      Waiting for application startup.
```

INFO: Application startup complete.

To stop the uvicorn server, press `Ctrl+C` in the terminal where it's running.

If you need to kill it from another terminal (if the process is running in the background), use:

```
# Find the process on port 8000 and kill it
lsof -ti:8000 | xargs kill -9
```

To stop the uvicorn server, use one of these methods:

1. *If the terminal is in the foreground* — Press `Ctrl+C` in the terminal where uvicorn is running.
2. *If running in the background or from another terminal* — Kill the process on port 8000:

```
lsof -ti:8000 | xargs kill -9
```

3. *To stop all related services (PostgreSQL, Redis Stack, and the app):*

Redis Stack runs as a Docker image because `__Homebrew` does not provide a `'redis-stack'` formula that can be installed directly via `'brew install'__`.

The regular Homebrew Redis (`'brew install redis'`) is a minimal Redis server that does `__not__` include the `RedisSearch` module. The `'langchain-redis'` package's `'RedisSemanticCache'` depends on `RedisSearch's 'FT._LIST'` command, which is only available in the Redis Stack distribution.

There is no `'brew install redis-stack'` alternative – only a `'redis-insight'` GUI cask exists via Homebrew. The official installation method for Redis Stack on macOS is either:

1. `__Docker__` (the approach we're using): `'docker run -d --name cka-redis -p 6379:6379 redis/redis-stack:latest'`
2. `__Direct download__` from `redis.com`: Download and run the Redis Stack server binary manually

So Docker is the simplest and cleanest way to get Redis Stack running locally. If you'd like to avoid Docker for Redis, there are two alternatives:

`__Option A: Disable Redis semantic caching__` – Remove or comment out the `'RedisSemanticCache'` block in `'app/rag.py'` (lines 39-45). The app will work without caching, just without the performance benefit of cached responses for semantically similar questions.

`__Option B: Use a simpler Redis cache__` – Replace `'RedisSemanticCache'` with a basic TTL-based Redis cache that doesn't require `RedisSearch`.

Would you like me to proceed with either of these alternatives to eliminate the Docker

dependency?

```
# Stop the app
lsof -ti:8000 | xargs kill -9

# Stop Redis Stack container
docker run -d --name cka-redis -p 6379:6379 -p 8001:8001 redis/redis-stack:latest
docker stop cka-redis
docker rm cka-redis

# Stop PostgreSQL (optional)
brew services stop postgresql@18
```

4.16. Open the Web UI

Open your browser and go to:

```
http://localhost:8000
```

You should see the **Company Knowledge Assistant** interface.

4.17. Ingest the Documents

In the browser UI, click the **"Ingest Data"** button.

Or use curl from the terminal:

```
curl -X POST http://localhost:8000/ingest
```

Check the ingestion status:

```
curl http://localhost:8000/ingest/status
```

You should eventually see:

```
{"ok":true,"status":"succeeded","started_at":...,"finished_at":...,"stats":{"documents":8,"chunks":300}}
```

Troubleshooting: If ingestion fails with a database error, double-check that: - PostgreSQL is running: `pg_isready` - The `vector` extension is enabled: `psql -U postgres -d postgres -c "SELECT * FROM pg_extension WHERE extname='vector';"` - The `langchain_pg_embedding` table exists: `psql -U postgres -d postgres -c "\d langchain_pg_embedding"` - Your `DATABASE_URL` in `.env` matches your local PostgreSQL credentials

4.18. □ Ask Questions

In the browser UI, type a question in the text area and click "Ask".

Or use curl:

```
curl -X POST http://localhost:8000/ask \  
-H "Content-Type: application/json" \  
-d '{"question": "What is the company policy on remote work?"}'
```

Example questions you can try:

- "What is the maximum number of PTO days that can be carried forward?"
- "How do I set up my corporate email?"
- "What is the VPN policy for remote employees?"
- "Tell me about the Q1 2025 announcement"

4.19. □ (Optional) Run RAGAS Evaluation

After ingestion is complete and the app is running:

```
# Terminal 1  
cd Frontier-Engineer/rag-knowledge-management  
uvicorn app.api:app --host 0.0.0.0 --port 8000 --reload  
  
# Terminal 2  
cd Frontier-Engineer/rag-knowledge-management  
python -m app.eval_ragas
```

This runs the test Q&A pairs from `seed/qna_test.json` and prints evaluation metrics.

4.20. Clean Database and Re-run Ingestion

Step 1: Clean the Database

Delete existing data from the `langchain_pg_embedding` table (which stores all document chunks, embeddings, and metadata). Connect to PostgreSQL and run:

```
psql -U postgres -d postgres -c "TRUNCATE TABLE langchain_pg_embedding;"
```

Or, if you also want to remove the HNSW index and recreate everything from scratch:

```
psql -U postgres -d postgres -c "SELECT * FROM langchain_pg_embedding;"
psql -U postgres -d postgres -c "DROP TABLE IF EXISTS langchain_pg_embedding CASCADE;"
psql -U postgres -d postgres -f init-db/init.sql
```

The **TRUNCATE** approach is faster — it clears all rows but keeps the table structure and indexes intact.

*Step 2: Clear the Redis Cache (Optional)

If you want to also clear cached LLM responses from the semantic cache:

```
redis-cli FLUSHDB
```

Step 3: Re-run Ingestion

Start the app (if not already running):

```
uvicorn app.api:app --host 0.0.0.0 --port 8000 --reload
```

Then trigger ingestion:

```
curl -X POST http://localhost:8000/ingest
```

Monitor the status:

```
curl http://localhost:8000/ingest/status
```

Wait until you see **"status": "succeeded"** with document/chunk counts.

Step 4: Verification After Ingestion

Check that the table has data:

```
psql -U postgres -d postgres -c "SELECT COUNT(*) FROM langchain_pg_embedding;"
```

You should see the chunk count (e.g., ~300 rows).

Chapter 5. File-by-File Explanation

5.1. `docker-compose.yml`

Defines three Docker services orchestrated together:

- **postgres** (`pgvector/pgvector:pg17`):
 - PostgreSQL 17 pre-installed with the `pgvector` extension for vector storage.
 - Mounts `init-db/` to `/docker-entrypoint-initdb.d/` so `init.sql` runs automatically on first start.
 - Has a health check using `pg_isready`.
- **app**:
 - Built from the `Dockerfile` in the same directory.
 - Runs the FastAPI application via Uvicorn on port 8000.
 - Depends on PostgreSQL being healthy before starting.
 - Mounts the project directory into `/app` for hot-reloading during development.
 - Has a health check via `/health` endpoint.
- **redis** (`redis/redis-stack:latest`):
 - Redis Stack with RedisJSON, Redisearch, RedisTimeSeries modules.
 - Port 6379 for Redis, port 8001 for RedisInsight GUI.
 - Used for semantic caching of LLM responses.
 - Append-only persistence enabled.

Two named volumes (`pgdata`, `redis-data`) persist database and cache data across container restarts.

5.2. `Dockerfile`

Multi-stage build using `python:3.11-slim`:

- Installs system dependencies: `build-essential`, `curl`, `ca-certificates`.
- Sets `WORKDIR` to `/app`.
- Copies and installs Python packages from `requirements.txt` (layered for caching).
- Copies the `app/` and `data/` directories.
- Exposes port 8000.
- Default CMD: `uvicorn app.api:app --host 0.0.0.0 --port 8000 --reload`.



The `requirements.txt` file is referenced by the `Dockerfile` but is currently missing from the repository. You would need to create it with the necessary dependencies (see the Dependencies section below).

5.3. .env

Environment configuration file containing:

Variable	Purpose
DATABASE_URL	PostgreSQL async connection string (postgresql+asyncpg://postgres:postgres@postgres:5432/postgres)
OPENAI_API_KEY	OpenAI API key for embeddings and LLM
LANGCHAIN_TRACING_V2	Enable LangChain tracing
LANGSMITH_API_KEY	LangSmith API key for observability
LANGSMITH_PROJECT	LangSmith project name (cka)

5.3.1. ☐ Where LangSmith is Actually Used

LangSmith is **not called directly anywhere in the application code** (`app/api.py`, `app/rag.py`, `app/ingest.py`, `app/utils.py`, `app/eval_ragas.py`) — there is no `import langsmith` or explicit LangSmith SDK call in this project.

It is enabled purely through **environment variables** picked up automatically by LangChain's built-in tracing hook:

```
LANGCHAIN_TRACING_V2 = true
LANGSMITH_API_KEY=lsv2_pt_...
LANGSMITH_PROJECT=cka
```

When `LANGCHAIN_TRACING_V2=true` and a valid `LANGSMITH_API_KEY` are present in the environment, LangChain automatically instruments and ships traces of every chain/LLM/retriever invocation — including the full RAG pipeline built in `app/rag.py`'s `_build_chain()` (retriever → Cohere reranker → `create_stuff_documents_chain` → `create_retrieval_chain`) — to the LangSmith platform under the `cka` project, with **no explicit tracing code required** in any `.py` file.

In short: LangSmith tracing is "wired on" passively via `.env` config; if you want to inspect a given `/ask` call's retrieval + rerank + generation steps, view them in the LangSmith dashboard for project `cka` rather than in the application source code.



The `.env` file in this repo contains a live-looking `LANGSMITH_API_KEY` value committed to the project. This should be rotated and moved to a secrets manager rather than kept in a plaintext `.env` file (see the security hardening notes in [documentation/0-enterprise-rag.adoc](#)).

DATA_DIR	Directory containing source documents (data)
RETRIEVAL_K	Number of documents to retrieve (5)
CO_API_KEY	Cohere API key for reranking

DATA_DIR	Directory containing source documents (data)
REDIS_URL	Redis connection URL (redis://cka-redis:6379/0)

5.4. `app/api.py` — FastAPI Application

The main web server with the following endpoints:

5.4.1. `GET /`

Serves the static `index.html` frontend page.

5.4.2. `POST /ingest`

Starts document ingestion as a **background async task**.

- Uses an `asyncio.Lock` to prevent concurrent ingestion jobs.
- Returns HTTP 409 if ingestion is already running.
- The background job updates a shared `_ingest_last` dict tracking status (`idle`, `running`, `succeeded`, `failed`), timestamps, statistics, and error messages.

5.4.3. `GET /ingest/status`

Returns the current state of the ingestion process (status, stats, timestamps, error info).

5.4.4. `POST /ask`

Accepts a JSON body `{"question": "..."}` and runs the full RAG pipeline:

1. Calls `answer_with_docs_async()`.
2. By default filters results to the `"guides"` category (hard-coded but easy to change).
3. Returns `{answer, sources, contexts}`.

The application also: - Mounts static files at `/static`. - Logs execution time for each `/ask` request.

5.5. `app/ingest.py` — Document Ingestion Pipeline

The ingestion process follows these steps:

5.5.1. Step 1: Load Documents (`_load_docs()`)

- Recursively scans the `DATA_DIR` directory for supported file types.
- Uses category-specific loaders from LangChain community:
 - `.md` → `UnstructuredMarkdownLoader`

- `.pdf` → `PyMuPDFLoader`
- `.docx` → `UnstructuredWordDocumentLoader`
- `.txt` → `TextLoader`
- Tags each document with a `category` metadata field derived from its parent folder name (e.g., `data/guides/file.md` → `category="guides"`).
- Skips directories and hidden files.
- Errors on individual files are caught and logged without stopping the pipeline.

5.5.2. Step 2: Chunk Documents (`_chunk()`)

- Uses `RecursiveCharacterTextSplitter` from LangChain.
- `chunk_size=900` characters per chunk.
- `chunk_overlap=120` characters between consecutive chunks to maintain context continuity.

5.5.3. Step 3: Store Embeddings

- Connects to the PostgreSQL vector store via `get_vector_store()`.
- Generates embeddings for each chunk using OpenAI's `text-embedding-3-small`.
- Asynchronously stores the embeddings + metadata in the `langchain_pg_embedding` table.

5.5.4. Step 4: Create Vector Index (`_create_index()`)

- Creates an **HNSW** (Hierarchical Navigable Small World) index for efficient approximate nearest-neighbor search.
- Uses **cosine distance** as the similarity metric.
- Configuration: `m=16` (connections per node), `ef_construction=64` (index quality).

The function returns a dict with `{documents: N, chunks: N}` statistics.

5.6. `app/rag.py` — RAG Retrieval + Generation Chain

5.6.1. System Prompt

The LLM is instructed to: - Answer strictly based on the provided context. - Reply "I don't know" if the answer is not in the context. - Prefer the most recent policy by effective date when conflicts exist.

5.6.2. Chain Construction (`_build_chain()`)

1. **Vector Store** — Connect to PostgreSQL vector store.
2. **Retriever** — Configured with:
 - `k=5` documents retrieved (configurable via `RETRIEVAL_K` env var).
 - Optional metadata filter by category (e.g., only search "guides").

3. **Reranker** — Uses Cohere `rerank-multilingual-v3.0` to reorder the 5 retrieved chunks, keeping the top 3 (`top_n=3`). This improves retrieval precision.
4. **LLM** — OpenAI `gpt-4o-mini` for cost-effective, fast generation.
5. **Chain** — Uses LangChain's `create_stuff_documents_chain` (stuff all context into prompt) wrapped by `create_retrieval_chain`.

5.6.3. □ When Does Reranking Happen? Is it a Separate API Call?

Yes — Cohere reranking happens **after** vector retrieval but **before** the LLM ever sees the context, and it is a completely separate, external network call to Cohere's servers (distinct from both the OpenAI embeddings call and the OpenAI chat completion call).

Why it's a distinct API call

`CohereRerank(top_n=3, model="rerank-multilingual-v3.0")` is a LangChain wrapper around Cohere's own `POST /v1/rerank` REST endpoint. It requires its own API key (`CO_API_KEY`, read from the environment — see the `.env` table above) and talks to Cohere's infrastructure using the `cohere` Python SDK/HTTP client. This is entirely separate from OpenAI, which handles embeddings and answer generation.

In `_build_chain()`, the reranker is wired in via `ContextualCompressionRetriever`:

```
base_retriever = store.as_retriever(search_kwargs=search_kwargs)

compressor = CohereRerank(
    top_n=3,
    model="rerank-multilingual-v3.0",
)

retriever = ContextualCompressionRetriever(
    base_retriever=base_retriever,
    base_compressor=compressor,
)
```

Step-by-step order of API calls for a single `/ask` request

1. **OpenAI Embeddings API** — the user's question is embedded (`text-embedding-3-small`) so it can be compared against stored vectors.
2. **pgvector similarity search** — `base_retriever` queries PostgreSQL/pgvector and returns the top `k` (default 5) most similar chunks. No LLM/reranker call is involved here — it's pure vector math inside Postgres.
3. **Cohere Rerank API** — `ContextualCompressionRetriever` calls `compressor.compress_documents(documents, query)`, which sends the question text plus the 5 candidate chunks to Cohere's rerank model. Cohere scores each chunk's relevance to the actual question and returns them reordered; LangChain keeps only the top 3 (`top_n=3`).
4. **OpenAI Chat Completion API** — only after reranking completes does `create_stuff_documents_chain` insert the reranked top-3 chunks into the `PROMPT` and call `gpt-4o-`

`mini` to generate the final answer.

So the full sequence is:

```
User Question
┆
OpenAI Embedding API (embed the question)
┆
pgvector similarity search (k=5 candidates from Postgres)
┆
Cohere Rerank API (separate external call → top_n=3)
┆
Prompt assembly (stuff top-3 chunks into PROMPT)
┆
OpenAI Chat Completion API (gpt-4o-mini generates the answer)
┆
Final Answer
```

Key takeaway: Reranking is **not** a manual step you'd find as separate code elsewhere in `rag.py` — it's encapsulated inside the single `retriever` object (`ContextualCompressionRetriever`) that `create_retrieval_chain` invokes. That one `retriever.invoke()` call internally performs both the vector search step and the Cohere rerank network call, in that order, before the doc chain ever runs the LLM. A single `/ask` request therefore triggers three distinct external API round-trips in sequence: OpenAI (embeddings) → Cohere (rerank) → OpenAI (chat completion).

5.6.4. Semantic Caching (DISABLED)

- `RedisSemanticCache` was previously used to cache LLM responses based on embedding similarity between questions, with `distance_threshold=0.98`.
- **Bug found:** a `distance_threshold` of `0.98` is far too permissive. In practice this caused **completely unrelated questions** (e.g. a PTO question and a travel-policy question) to be treated as "similar enough," so the cache incorrectly returned a previously cached answer for a different question entirely — silently giving users the wrong answer.
- **Fix:** semantic caching has been **disabled** in `app/rag.py` via `set_llm_cache(None)`. The `RedisSemanticCache` import/setup was removed. Every `/ask` call now always generates a fresh answer from the LLM using the freshly retrieved context, eliminating the risk of stale/incorrect cached answers.
- Redis is therefore **no longer required** for the app to function correctly. It was previously used only for this semantic cache.
- If semantic caching is reintroduced in the future, use a much stricter `distance_threshold` (e.g. `0.05–0.15`, since `langchain-redis` treats this value as a **distance** where lower = stricter/more-similar match required) and thoroughly test with dissimilar questions before relying on it in production.

5.6.5. `answer_with_docs_async()`

- Builds the chain, invokes it asynchronously with the user's question.
- Extracts and returns:
 - `answer` — The generated text.
 - `sources` — Unique source file paths from retrieved documents.
 - `contexts` — Full text content of the retrieved chunks.

5.7. `app/utils.py` — Database & Vector Store Setup

- Reads `DATABASE_URL` from environment.
- Creates a `PGEngine` (from `langchain_postgres`) for managing the PostgreSQL connection.
- Initializes OpenAI embeddings model (`text-embedding-3-small`, 1536 dimensions).
- `get_vector_store()` creates and returns an `AsyncPGVectorStore` configured with:
 - `table_name="langchain_pg_embedding"` — the vector storage table.
 - `metadata_json_column="langchain_metadata"` — JSON metadata column.
 - `metadata_columns=["category"]` — allows filtering searches by category.

5.8. `app/eval_ragas.py` — RAGAS Evaluation

A script that evaluates the RAG system's quality using the **RAGAS** framework:

- Loads test Q&A pairs from `seed/qna_test.json`.
- For each question, sends a POST to `http://localhost:8000/ask` and captures the answer and retrieved contexts.
- Creates `SingleTurnSample` objects with: `user_input`, `response`, `retrieved_contexts`, `reference`.
- Evaluates using four RAGAS metrics:
 - **Faithfulness** — Is the answer factually consistent with the retrieved context?
 - **Answer Relevancy** — How relevant is the answer to the question?
 - **Context Precision** — How relevant are the retrieved documents to the question?
 - **Context Recall** — Are all relevant documents retrieved?
- Outputs per-question scores and averages.

5.9. `app/static/index.html` — Frontend UI

A clean, single-page HTML application with:

- A header showing the brand name and an **"Ingest Data"** button with status pill.
- A text area for entering questions (supports `Ctrl+Enter` / `Cmd+Enter` shortcut).
- **"Ask"** and **"Clear"** buttons.

- An answer display area with source document links.
- JavaScript that:
- Calls `/ingest` (POST) to trigger document ingestion.
- Polls `/ingest/status` every 2 seconds to update the status pill.
- Calls `/ask` (POST) with the user's question.
- Renders the answer and sources in the UI.

5.10. `app/static/style.css`

Clean, minimal styling with:

- Responsive layout, max-width container.
- Pill-style status indicators (idle=grey, running=blue, succeeded=green, failed=red).
- Card-based UI components.
- Accessible button states (hover, disabled).

5.11. `init-db/init.sql`

PostgreSQL initialization script that runs automatically when the Postgres container starts for the first time:

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE IF NOT EXISTS langchain_pg_embedding (
  langchain_id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  content text,
  embedding vector(1536),
  langchain_metadata jsonb,
  category text
);
```

- Enables the `vector` extension (required by pgvector).
- Creates the `langchain_pg_embedding` table with:
- `langchain_id` — UUID primary key.
- `content` — The raw text of the chunk.
- `embedding` — 1536-dimensional vector (OpenAI `text-embedding-3-small` dimension).
- `langchain_metadata` — JSONB for flexible metadata.
- `category` — Direct column (also indexed via `metadata_columns` in LangChain).

5.11.1. What is `langchain_metadata` Used For?

`langchain_metadata` is a `jsonb` column in the `langchain_pg_embedding` table that stores **arbitrary, flexible per-chunk metadata as JSON**, separate from the actual embedding vector and raw text content columns.

Where it's defined

- `init-db/init.sql: langchain_metadata jsonb` — declared as a JSONB column in the table schema.
- `app/utils.py (get_vector_store())`: passes `metadata_json_column="langchain_metadata"` to `AsyncPGVectorStore.create()`, telling LangChain which column to use for reading/writing metadata.

What goes into it

During ingestion (`app/ingest.py`), every `Document` object loaded from a file (PDF/MD/DOCX/TXT) carries a `.metadata` dict. For example:

```
d.metadata["category"] = category    # e.g. "guides", "policies"
```

Document loaders such as `PyMuPDFLoader` also automatically attach fields like `source` (the file path) and page numbers. When `store.aadd_documents(chunks)` runs during ingestion, LangChain serializes each chunk's entire `.metadata` dict into JSON and writes it into the `langchain_metadata` column — one JSON blob per row/chunk.

Why it's useful

1. **Flexible schema** — new metadata fields (source, page, author, date, etc.) can be added without altering the table schema, since it's just JSON.
2. **Citations/traceability** — in `app/rag.py`'s `answer_with_docs_async()`, each retrieved document's `d.metadata.get("source")` is read to build the `sources` list returned to the user, so they can see which file(s) an answer came from. This value comes straight out of the `langchain_metadata` JSON blob.
3. **Filtered search support** — `metadata_columns=["category"]` in `utils.py` promotes the `category` key specifically out of the JSON blob into its own indexed `category` SQL column (separate from `langchain_metadata`), enabling fast `WHERE category = 'guides'` filtering during vector search (see `_build_chain()`'s `search_kwargs["filter"] = {"category": category}`).

Summary: `langchain_metadata` is the general-purpose JSON "extra info" bucket for each embedded chunk (source file, page, etc.), while frequently-filtered fields like `category` are additionally promoted to their own real column for efficient querying — both are populated automatically by LangChain's vector store during ingestion.

5.12. seed/qna_test.json

A JSONL file with two test Q&A pairs for evaluation:

1. "What is the maximum number of PTO days that can be carried forward to the next year?" → Answer: 6 days.
2. "What is the company's policy on remote work?" → Answer: Role-based approval, home workstation requirements, core hours 10:00–17:00 IST.

5.13. data/ — Sample Documents

Pre-loaded company documents organized by category:

Category	Files	Format
announcements	merger-news-2024.pdf q1-announcement-2025.pdf	PDF
faqs	travel-faq.txt vpn-faq.md	TXT, MD
guides	email-setup.docx jira-guide.pdf vpn-setup.md	DOCX, PDF, MD
handbooks	employee_handbook_2025.docx	DOCX
policies	Expense Policy.pdf PTO Policy.pdf	PDF

The category is derived from the folder name and stored as metadata for filtered retrieval.

5.14. pgvector

pgvector is the PostgreSQL extension that powers the entire vector storage and semantic search in this project. Here's where it's used:

Infrastructure Layer

- **docker-compose.yml**: The PostgreSQL container uses the **pgvector/pgvector:pg17** image which comes with pgvector pre-installed.
- **init-db/init.sql**: Enables the **vector** extension in PostgreSQL on database creation.
- **PostgreSQL table langchain_pg_embedding**: This is the actual table created by **AsyncPGVectorStore** that stores:
 - The **embedding vectors** (1536-dimensional floats from **text-embedding-3-small**)
 - Document chunk **text content**
 - **Metadata** (category, source, etc.)

Application Layer — **app/utils.py - AsyncPGVectorStore** (line 46-61) is the Python abstraction over the pgvector-backed table. It handles: - Converting text to vectors via the embedding model - Storing vectors in PostgreSQL - Performing **cosine similarity search** using pgvector's internal operations

Ingestion — **app/ingest.py - store.aadd_documents(chunks)** (line 184): Embeds document chunks and stores them as vectors in pgvector. - **HNSW index creation** (lines 130-152): Creates a pgvector-compatible Hierarchical Navigable Small World index on the vector column to accelerate approximate nearest-neighbor search.

Retrieval — `app/rag.py` - `store.as_retriever()` (line 140): Creates a retriever that queries pgvector for the most semantically similar chunks to the user's question.

Data Flow Summary

User Question → OpenAI Embedding (1536-d vector) → pgvector similarity search → Top-k chunks → Cohere Rerank → LLM → Answer

So **pgvector** is the **vector database** that makes semantic search possible — without it, the RAG pipeline would have no way to find relevant document chunks for a given question.

5.15. Verifying Everything is Running

Use these commands to verify each component:

```
# Verify PostgreSQL
pg_isready

# Verify Redis
redis-cli ping

# Verify the app
curl http://localhost:8000/health

# Check if the app is serving
curl -s -o /dev/null -w "%{http_code}" http://localhost:8000/
```

5.16. Stopping the Application

To stop everything:

```
# Stop the FastAPI server – press Ctrl+C in the terminal where it's running

# Stop PostgreSQL (if you want to free resources)
brew services stop postgresql@17

# Stop Redis (if you want to free resources)
brew services stop redis

docker run -d --name cka-redis -p 6379:6379 -p 8001:8001 redis/redis-stack:latest

# Deactivate the virtual environment
```

deactivate

5.17. Restarting After Stopping

When you want to run the project again later:

```
# Start services
brew services start postgresql@17
brew services start redis

# Activate environment
cd Frontier-Engineer/week2/e2e-projects/rag-knowledge-management
source venv_cka/bin/activate

# Run the app
uvicorn app.api:app --host 0.0.0.0 --port 8000 --reload
```

Note: The data in PostgreSQL and Redis persists across restarts if you used `brew services`. You don't need to re-ingest documents unless you clear the database.

Chapter 6. Troubleshooting

6.1. ModuleNotFoundError: No module named 'langchain_classic'

Install the missing package:

```
pip install langchain-classic
```

6.2. ModuleNotFoundError: No module named 'langchain_postgres'

```
pip install langchain-postgres
```

6.3. Connection refused to PostgreSQL

Ensure PostgreSQL is running:

```
brew services list | grep postgresql  
pg_isready
```

If not running: `brew services start postgresql@17`

6.4. Connection refused to Redis

```
brew services list | grep redis  
redis-cli ping
```

If not running: `brew services start redis`

6.5. Ingestion fails with authentication error

Your PostgreSQL user might need a password or the correct permissions. Try:

```
# Set a password for postgres user  
psql -d postgres -c "ALTER USER postgres PASSWORD 'postgres';"  
  
# Then update .env to use password authentication  
# DATABASE_URL=postgresql+asyncpg://postgres:postgres@localhost:5432/postgres
```

Or if your macOS user is the PostgreSQL superuser:

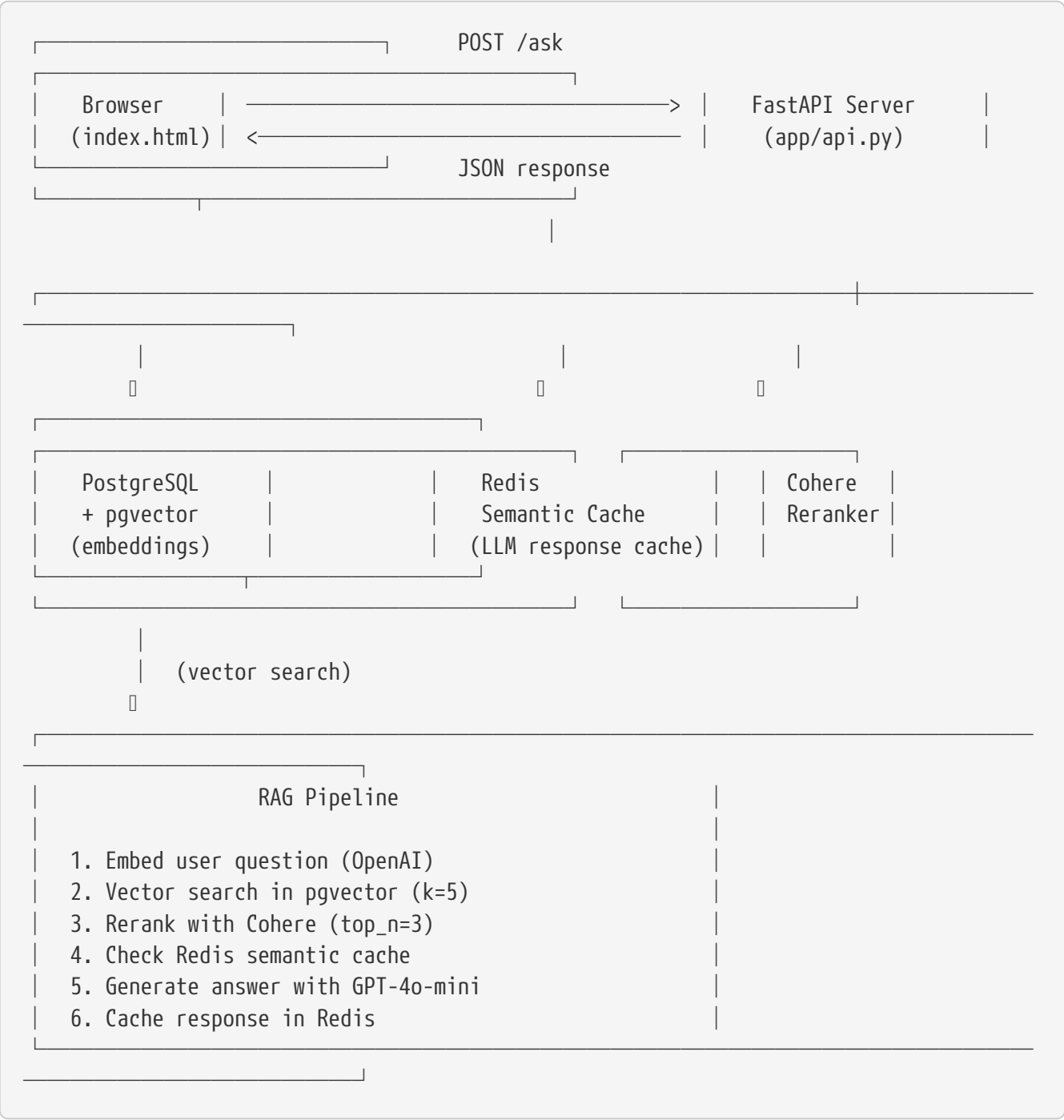
```
# Use trust authentication – update .env to:  
DATABASE_URL=postgresql+asyncpg://YOUR_USERNAME@localhost:5432/postgres
```

Where **YOUR_USERNAME** is the output of **whoami**.

Chapter 7. Summary of Services and Ports

Service	Port	Purpose
FastAPI App	8000	Web UI and API endpoints
PostgreSQL	5432	Vector database (embeddings + metadata)
Redis	6379	Semantic cache for LLM responses
RedisInsight	8001	Redis GUI (only if running redis-stack)

Chapter 8. Architecture Diagram



Chapter 9. Dependencies

The project requires these Python packages (should be in `requirements.txt`):

- **FastAPI / Uvicorn** — Web framework + ASGI server
- **LangChain ecosystem** — `langchain`, `langchain-openai`, `langchain-community`, `langchain-postgres`, `langchain-redis`, `langchain-cohere`, `langchain-classic`
- **OpenAI** — `openai` for embeddings and LLM
- **Document loaders** — `pypdf`, `pymupdf`, `unstructured`, `python-docx`
- **Database** — `psycopg2-binary`, `asyncpg`
- **Cache** — `redis`
- **Reranking** — `cohere`
- **Evaluation** — `ragas`, `pandas`
- **HTTP** — `httpx` (for async HTTP calls)
- **Other** — `python-multipart` (for form data parsing)

Chapter 10. Key Design Decisions

1. **Async everywhere** — All I/O operations (DB queries, API calls, file loading) are asynchronous using `asyncio`, ensuring the server remains responsive under concurrent requests.
2. **Background ingestion** — Document ingestion runs as a background task, so the API remains available during processing. A lock prevents concurrent ingestion jobs.
3. **Metadata filtering** — Documents are categorized by source folder, enabling filtered retrieval by category (e.g., only search "policies" or "guides").
4. **Reranking** — Using Cohere's reranker after initial vector retrieval improves precision by re-ranking the top-k results, ensuring only the most relevant contexts are fed to the LLM.
5. **Semantic caching (DISABLED)** — Redis Semantic Cache was originally added to avoid redundant LLM calls for semantically similar questions. However, a `distance_threshold=0.98` proved far too permissive and caused unrelated questions (e.g. PTO vs. travel policy) to incorrectly return each other's cached answers. Semantic caching has since been **disabled** (`set_llm_cache(None)` in `app/rag.py`) so every question is always answered fresh — correctness was prioritized over caching performance.
6. **Category-based retrieval** — The `/ask` endpoint currently hard-codes `category="guides"`, demonstrating filtered search. This can be parameterized for production use.
7. **Evaluation framework** — RAGAS integration allows quantitative measurement of RAG quality, essential for iterating on the pipeline.

Chapter 11. Notes

- The `requirements.txt` file is referenced by the `Dockerfile` but is currently **missing** from the repository. You must create it before building/running the Docker container.
 - The `.env` file contains **live API keys** — in production, use a secrets manager or Docker secrets instead.
 - The `/ask` endpoint currently filters to the `"guides"` category. To search across all categories, pass `category=None` in `rag.py`.
 - The `data/` directory contains sample documents. Replace them with your actual company documents for a production deployment.
-

Chapter 12. Q&A

Briefly explain the use of **PGVectorStore**

PGVectorStore Usage in `utils.py`

The file uses `AsyncPGVectorStore` (from `langchain_postgres`) to provide a PostgreSQL-backed vector database for the RAG knowledge management system. Here's how it's used:

1. Database Connection Setup (lines 18-24)

- Reads the `DATABASE_URL` environment variable (a PostgreSQL connection string using the `asyncpg` driver).
- Creates a `PGEngine` from this connection string, which manages the async database connection pool.

2. Embedding Model Initialization (line 29)

- Initializes OpenAI's `text-embedding-3-small` model to convert text into vector embeddings.

3. `get_vector_store()` Factory Function (lines 33-63)

- An async factory function that creates and returns an `AsyncPGVectorStore` instance configured with:
- `engine`: The PostgreSQL connection engine.
- `embedding_service`: The OpenAI embedding model for vector generation.
- `table_name`: `"langchain_pg_embedding"` — the database table that stores the vector embeddings.
- `metadata_json_column`: `"langchain_metadata"` — a JSON column for storing arbitrary document metadata alongside each vector.
- `metadata_columns`: `["category"]` — specific metadata fields extracted from the JSON column that can be used as filters during similarity search.

Purpose in the RAG System

The `AsyncPGVectorStore` serves as the *vector storage and retrieval layer* for the RAG pipeline. It:

- Stores document chunks as vector embeddings in PostgreSQL (using the `pgvector` extension).
- Enables *semantic similarity search* — given a query, it finds the most semantically relevant document chunks by comparing vector distances.
- Supports *metadata filtering* (by `category`) to narrow down search results.
- Uses *async operations* for non-blocking database access, suitable for the FastAPI application.

Ingestion

You're exactly right. Since you've already run ingestion, the vector embeddings and document metadata are stored **persistently** in the PostgreSQL database via the `pgvector` extension. The `AsyncPGVectorStore` plays two distinct roles here:

1. **Ingestion Phase** (run once) — via `ingest.py`

- Documents are split into chunks → embedded → stored into the `langchain_pg_embedding` table using `AsyncPGVectorStore`.
- This populates the database with vectors and metadata (e.g., category).

2. **Query/Retrieval Phase** (run many times) — via `rag.py` / `api.py`

- The same `AsyncPGVectorStore` (created by `get_vector_store()` in `utils.py`) connects to the **already-populated** `langchain_pg_embedding` table.
- It performs **similarity search** against the stored vectors — no re-embedding or re-storing of documents needed.
- The database persists the data across application restarts.

So the `PGVectorStore` provides **persistent, reusable storage** — you ingest once and query many times. The `AsyncPGVectorStore.create()` call in `utils.py` is idempotent in the sense that if the table already exists, it simply connects to it rather than recreating it (depending on the underlying LangChain implementation with `sync_mode="OFF"` and `create_extension=False` or similar flags). If you want to be explicit about not recreating the table, you could also use `AsyncPGVectorStore(engine=PG_ENGINE, ...)` directly instead of `.create()`.
